FF

FrameForge Studio

LOCAL VIDEO STUDIO

FrameForge Studio

Guide & Operations Manual

Version 0.1.0 · Electron desktop application · Windows, macOS, Linux

Rendered on this machine · 2026-11-10 · No backend server

Contents

01 What FrameForge Studio is 3

02 How a render is put together 3

03 System requirements 4

04 Install and first run 4

05 The interface 5

06 API keys 6

07 Making your first video 8

08 Writing shots 8

09 Providers, cost and timing 9

10 Files and folders 10

11 Checking your installation 10

12 Troubleshooting 12

13 Security and privacy 13

14 Distributing the application 13

15 Environment variables 14

16 Glossary 14

1 What FrameForge Studio is

FrameForge Studio is a desktop application for producing short AI-generated videos. You describe the video as a set of shots, choose a video generation provider, and the app renders a finished MP4 on your own machine.

Three things define the product:

1. **There is no FrameForge server.** The application never contacts an endpoint we

operate. It talks only to the video provider you choose, using your account.

1. **You supply the API keys.** Every generation is billed by the provider directly to

you. Keys are stored in your operating system's credential store.

1. **Rendering happens locally.** Assembling, titling and encoding all run through

FFmpeg on this computer. Nothing is uploaded for rendering.

What it is not: a hosting service, a stock-footage library, a timeline editor, or a billing relationship. There is no account to create, no subscription, and no usage reporting.

2 How a render is put together

Every job runs the same seven stages, in order. The Jobs screen shows each one live.

#	STAGE	WHAT HAPPENS	TYPICAL TIME
1	Validate inputs & toolchain	Checks FFmpeg is installed and a key exists for the chosen provider	instant
2	Storyboard via OfficeCLI	Writes the production brief — Markdown from your own LLM when one is configured, otherwise a `.docx` from OfficeCLI	5–60 s
3	Generate shots with provider	Submits each shot, then polls until the provider returns a video	1–10 min per shot
4	Download source clips	Streams the returned clips into the job folder	seconds
5	Compose title cards	Renders opening and closing cards, one PNG per frame	10–20 s
6	Render with FFmpeg	Normalises every segment, joins them, writes the MP4	10–60 s
7	Export deliverables	Writes the poster frame, optional `.pptx` deck, and file records	seconds

Stages 5 and 6 are the parts that always run locally, even if you skip the documents. A job is never partly uploaded: if the app closes mid-render, the job is marked interrupted on next launch and the files that already exist are left on disk.

The video is assembled as: opening card → each generated shot → closing card, joined with either hard cuts or a crossfade. Card durations are 3.2 s (2.6 s in portrait) for the opening and 2.8 s (2.2 s in portrait) for the closing.

---

3 System requirements

REQUIREMENT	DETAIL
Operating system	Windows 10/11, macOS 12+, or a modern Linux desktop
Node.js	22 or newer (only needed to build or run from source)
pnpm	10 or newer
FFmpeg	`ffmpeg` and `ffprobe` on `PATH`, or set `FRAMEFORGE_FFMPEG_DIR`
OfficeCLI	Optional; bundled as an npm dependency. Document stages are skipped without it
Disk	Roughly 40 MB per 10 s of 1080p output, plus the source clips
Network	Outbound HTTPS to your chosen provider only

FFmpeg 6 or newer is recommended. The build is verified against FFmpeg 9.x on Windows.

---

4 Install and first run

From source

```
pnpm install
pnpm dev
```

`pnpm install` downloads the Electron runtime and unpacks the OfficeCLI binary. If your network blocks either, see §12.

Production build

```
pnpm build          # typecheck + bundle main, preload and renderer
pnpm package        # unpacked application in release/
pnpm dist           # platform installer (NSIS / DMG / AppImage)
```

First launch

1. The window opens on **Studio**.
2. Check the status pills in the header:

- **FFmpeg** — must be green. Red means FFmpeg was not found. - **OfficeCLI ready** — amber if missing; only document stages are affected. - **OS keychain** — green when an OS-backed vault is available.

1. Open **Keys** and add the key for at least one provider (§6).
2. Return to **Studio** and press **Render locally**.

---

5 The interface

The application has four sections, reachable from the header.

Studio

The authoring surface, in two columns.

- **Left — the composer.** Project title and output name, provider and model, aspect

ratio, shot length, transition, the shot list, title-card copy and accent colour, and the Office deliverables toggles. The bar at the bottom shows the cost estimate and the **Render locally** button.

- **Right — the composition preview.** The opening card as it will render, drawn from

the same HTML that the encoder captures. Below it, the render plan and the status of the most recent job.

The preview updates about half a second after you stop typing. It is a preview of the *opening card*, not of the generated footage — provider clips do not exist until the job runs.

Jobs

The render queue on the left, the selected job on the right.

- Each row shows the title, status, progress bar, provider and age.
- The detail panel shows all seven stages with their status and a one-line result, the

artifact list with sizes, and a **Show in folder** action per artifact.

- **Cancel** stops a running job at its next checkpoint. Clips already downloaded are

kept.

Jobs are stored as JSON files and survive restarts. One job runs at a time; additional jobs queue in order.

Keys

One card per provider, showing whether a key is configured, its last four characters and a short fingerprint, plus **Test**, **Remove** and the storage explanation.

System

The runtime versions, toolchain discovery results (with paths), vault backend, the exact list of hosts the app may contact, and where your workspace lives.

---

6 API keys

Where keys come from

Create a key in the provider's own dashboard. FrameForge never proxies, resells or inspects your usage.

PROVIDER	CAPABILITY	KEY FORMAT	DEFAULT MODEL	ALSO AVAILABLE
Runway	text-to-video	`key_…`	`gen4_turbo`	`gen3a_turbo`
Luma Dream Machine	text-to-video	vendor token	`ray-2`	`ray-flash-2`
fal.ai	text-to-video	`id:secret`	`fal-ai/kling-video/v2/master/text-to-video`	`fal-ai/minimax/hailuo-02/standard/text-to-video`, `fal-ai/veo3`
Replicate	text-to-video	`r8_…`	`minimax/video-01`	`google/veo-3-fast`, `tencent/hunyuan-video`
HeyGen	avatar video	vendor token	default avatar	any avatar id, `avatarId:voiceId`
OpenAI Sora	text-to-video	`sk-…`	`sora-2`	`sora-2-pro`

Each provider card links to its documentation.

Universal LLM (DeepSeek, OpenRouter, Groq, Ollama…)

The **Custom / Universal provider** card accepts any OpenAI-compatible chat endpoint, so the authoring side of the app is not limited to the six video providers. It takes four things:

FIELD	EXAMPLE
Provider name	`DeepSeek`
Base URL	`https://api.deepseek.com/v1`
Model ID	`deepseek-chat`
API key	your key for that service

Presets fill the first three for DeepSeek, OpenRouter, Groq and a local Ollama, and any other endpoint that speaks `POST /v1/chat/completions` works the same way.

This provider does **not** generate video — it writes the storyboard brief and can rewrite your shot prompts before they are sent to the video provider. It is stored exactly like the others: name, base URL and model go to `settings.json`, the key goes to the OS credential store under `provider:custom`, and **Test** performs one authenticated `GET /models` so a wrong key is caught without spending tokens.

The app builds the HTTP client for this provider from your base URL alone: that single host is allowlisted for that client, so no other address becomes reachable. Loopback addresses are permitted only because you typed them (useful for Ollama or LM Studio on your own machine).

How keys are stored

- The key is written to the operating system's credential store: **Windows Credential

Manager, macOS Keychain, **or** Secret Service** (libsecret) on Linux. The service name is `FrameForge Studio` and the account name is `provider:<id>`.

- If the native keyring module cannot load, the app falls back to Electron `safeStorage`,

which uses DPAPI on Windows and the Keychain on macOS. On Linux it refuses the `basic_text` backend, because that is not encryption.

- Only three non-secret facts are written to disk in `vault/keys.json`: the last four

characters, the first 12 hex characters of a SHA-256 fingerprint, and a timestamp.

What the app refuses

- Keys shorter than 8 characters, longer than 4096, containing spaces, tabs or control

characters, or wrapped in quotes. Paste the raw value.

- Storing a key when no OS-backed encryption is available: the app falls back to

session-only memory and says so in the interface.

Rotating and removing

Remove deletes both the credential-store entry and the metadata. To rotate, remove then store the new key; the fingerprint and last four characters change, which is how you confirm the swap.

Test performs a single read-only request against the provider and reports latency. It treats a 401 or 403 as a rejected key and any other status as accepted — it never starts a generation, so testing costs nothing.

---

7 Making your first video

1. **Title** — internal name, shown in Jobs.
2. **Output file** — the filename stem for the MP4. Letters, numbers, spaces and dashes.
3. **Aspect** — 16:9 (1920×1080), 9:16 (1080×1920) or 1:1 (1080×1080).
4. **Provider and model** — see §6. Switching provider resets the model to its default.
5. **Shot length** — 4, 5, 6, 8 or 10 seconds per shot.
6. **Join** — Crossfade or Hard cut.
7. **Shots** — one prompt per shot, up to six. See §8.
8. **Title cards** — headline (80 characters), subhead (120), accent colour. Switch

Show titles off to render without cards.

1. **Office deliverables** — enable the storyboard brief and/or the delivery deck. With a

universal LLM configured, the brief is Markdown written by your model; without one it falls back to OfficeCLI.

1. **Authoring model** — the *Rewrite each shot prompt* switch asks your LLM to turn every shot

into a camera + subject + light + grade sentence before generation. The before/after pairs are saved as `enhanced-prompts.json` next to the job.

1. Review the preview, then **Render locally**.

If the button is disabled, the reason is stated directly beneath it: no FFmpeg, no key for the chosen provider, or an empty shot prompt.

What you get

Written to the output folder (§10):

- `<output-name>.mp4` — H.264, `yuv420p`, 30 fps, `+faststart`, no audio track.
- `<output-name>-poster.jpg` — a frame from 1.5 s in (or mid-film for short pieces).
- `<output-name>` storyboard `.docx` and delivery `.pptx` when requested.

---

8 Writing shots

The prompt is the whole direction. A shot that names camera, subject, light and mood generates far more predictably than a shot that names a concept.

A workable pattern:

> *[camera movement] + [subject and action] + [environment] + [lighting] + [grade or texture]*

Example:

> Slow dolly through a sun-drenched design studio, industrial concrete and warm wood, > soft volumetric light, shallow depth of field, cinematic 35mm

Practical notes:

- **One idea per shot.** Two subjects or a scene change in five seconds reads as mush.
- **Name the camera.** "Slow orbit", "macro detail", "locked-off wide", "handheld

follow" all behave differently.

- **Keep brand-critical text out of the footage.** Providers render lettering poorly;

put copy in the title cards, which are vector-crisp and editable.

- **Match shots by grade, not by subject.** Repeating "soft morning light, warm neutral

grade" across shots is what makes unrelated clips feel like one film.

- **Six shots at 5 s is a 30-second piece** before crossfades trim it. Cost scales

linearly with shot count.

HeyGen is different: the prompt is spoken by an avatar, so write it as narration and set the model field to your avatar id (`avatarId` or `avatarId:voiceId`).

---

9 Providers, cost and timing

The estimate in the action bar uses per-second list prices and shows what one run should cost. It is an estimate, not a quote: providers bill your account directly, and their prices change.

PROVIDER	ESTIMATED COST PER SECOND OF OUTPUT
Runway	$0.10
Luma	$0.09
fal.ai	$0.07
Replicate	$0.06
HeyGen	$0.05
OpenAI Sora	$0.10

Generation is the slow stage. The app submits a shot, then polls every 4 seconds for up to 20 minutes per shot. Progress is reported per shot, so a six-shot job shows six plateaus rather than one smooth bar.

Duration handling differs slightly by provider — some accept only fixed lengths. The app maps your choice to the nearest supported value rather than failing the job.

---

10 Files and folders

Everything lives under the application's data directory:

PLATFORM	DATA DIRECTORY
Windows	`%APPDATA%\frameforge-studio`
macOS	`~/Library/Application Support/frameforge-studio`
Linux	`~/.config/frameforge-studio`

Inside it:

PATH	CONTENTS
`workspace/output/`	Final MP4s and posters — this is the folder you deliver from
`workspace/projects/<job-id>/clips/`	Downloaded shots and the encoded cards
`workspace/projects/<job-id>/frames/`	Per-frame PNGs for the title cards
`workspace/projects/<job-id>/docs/`	Storyboard `.docx`, delivery `.pptx`
`jobs/<job-id>.json`	Job records, including stage history and artifact paths
`vault/keys.json`	Key metadata only — never secret material

Frame PNGs are kept so a render can be reproduced frame-for-frame; delete the project folder when a job is finished if disk space matters.

---

11 Checking your installation

Four commands, in increasing scope:

```
pnpm typecheck      # strict TypeScript, no `any`
pnpm test           # 55 tests: vault, providers, LLM, FFmpeg, pipeline, jobs, documents
pnpm smoke          # boots the real app, checks the bridge and vault, writes smoke.png
pnpm smoke:job      # full render through the UI against a local mock provider
```

`smoke:job` needs no provider account. It generates a test clip with FFmpeg, serves it from a local mock, drives the application through a two-shot job in the real interface, then verifies the exported file with ffprobe. Use it after any change to confirm the toolchain still works end to end.

`smoke` also round-trips a throwaway key through the real credential store and removes it again, so a passing run proves persistence, not just the IPC path.

---

12 Troubleshooting

SYMPTOM	CAUSE	FIX
Header pill shows **FFmpeg missing**; render button disabled	`ffmpeg` / `ffprobe` not on `PATH`	Install FFmpeg, or set `FRAMEFORGE_FFMPEG_DIR` to the folder containing both binaries, then restart
Stage 1 fails with "No API key stored"	No key for the selected provider	Add the key in **Keys**; **Test** it
Test reports "Provider rejected the key (401)"	Wrong key, revoked key, or the key belongs to a different account	Regenerate in the provider dashboard
Generation fails with 429	Provider rate limit	Wait, or reduce concurrent work elsewhere on the same account
Shot times out after 20 minutes	Provider queue is slow or the request stalled	Retry the job; try a shorter shot or a different model
Download fails with "Refused media URL pointing at a private address"	Provider returned a URL resolving to an internal address	Treated as unsafe by design; retry, and report it if it persists
Preview panel is dark	Composition not yet built, or the frame was never measured	Wait a second; re-open Studio. If it persists, run `pnpm smoke` and inspect the capture
Job fails with `EPERM … rename` on Windows	A file in the workspace is locked by another program	The app retries automatically; if it still fails, close indexers/antivirus scans on the data directory
Only document stage fails	OfficeCLI exited non-zero	Run `officecli --version` and `officecli whoami` in a terminal. First use needs a network download of its binary
Credential store unavailable; keys vanish on restart	No keyring on the machine (common on bare Linux)	Install libsecret / a keyring daemon, or accept session-only storage
OfficeCLI fails to unpack on Windows from git-bash	MSYS `tar` treats `C:` as a remote host	Start the app from a normal shell, or run `officecli --version` once from PowerShell to unpack the binary
`pnpm install` did not download Electron	Network/proxy blocked the binary download	Set the standard Electron mirror environment variables and reinstall

When a job fails, the failing stage carries the provider's own message. That text is the most precise diagnostic available — read it before changing anything.

13 Security and privacy

Keys. Stored in the OS credential store. Never sent anywhere except the provider that owns them, in an `Authorization` header from the main process. They are never returned to the interface — the renderer can set, test and delete a key, but cannot read it back.

Network. The renderer cannot make arbitrary requests. Outbound API calls are restricted to six hosts, enforced in one place:

```
api.dev.runwayml.com · api.lumalabs.ai · queue.fal.run
api.replicate.com · api.heygen.com · api.openai.com
```

A custom LLM adds exactly one more host: the one in your base URL, and only for the client built for that provider — it is not merged into the app-wide allowlist.

Media downloads must be HTTPS and may not resolve to loopback, RFC1918, link-local, `.local` or `.internal` addresses, because providers return signed CDN URLs that cannot be pre-allowlisted.

The window. The renderer runs sandboxed with context isolation, no Node integration, a strict Content-Security-Policy (`default-src 'none'`), denied navigation, denied pop-ups, and denied permission requests other than clipboard writes.

Child processes. FFmpeg and OfficeCLI are spawned with argument arrays, never through a shell, so a file name or prompt cannot become a command.

Files. Writes are atomic. Job records are reconciled on launch, so a crash cannot leave a job stuck in "running".

Telemetry. None. There is no analytics, crash reporting or update ping.

---

14 Distributing the application

```
pnpm package    # unpacked app for smoke-testing
pnpm dist       # platform installers
```

`@napi-rs/keyring` and `officecli` are unpacked from the asar archive because they contain native executables; this is configured in `package.json` under `build.asarUnpack`. Installers are unsigned by default — signing and notarisation belong in your release pipeline.

---

15 Environment variables

VARIABLE	PURPOSE
`FRAMEFORGE_FFMPEG_DIR`	Folder containing `ffmpeg` and `ffprobe`
`FFMPEG_DIR`	Fallback for the same
`FRAMEFORGE_OFFICECLI`	Explicit path to the `officecli` binary
`FRAMEFORGE_DEV_PROVIDER`	Provider id to redirect (testing)
`FRAMEFORGE_DEV_BASE_URL`	Base URL for that provider, e.g. a local gateway or mock
`FRAMEFORGE_SMOKE`	Enable the self-check harness
`FRAMEFORGE_SMOKE_OUT`	Where the self-check writes its screenshot
`FRAMEFORGE_SMOKE_TAB`	Section to open before capturing (`keys` , `jobs` , `system`)
`FRAMEFORGE_SMOKE_JOB`	Run a full job through the UI during the self-check
`FRAMEFORGE_SMOKE_LLM_URL`	Point the self-check at an OpenAI-compatible mock, so the storyboard and prompt-enhancement paths are exercised too

`FRAMEFORGE_DEV_*` exist so the desktop app can be pointed at a mock or a self-hosted gateway during testing. Leave them unset in normal use — they also relax the loopback rule for that one provider.

16 Glossary

Aspect — output frame shape: 16:9, 9:16 or 1:1. **Artifact** — any file a job produced: clip, composition, video, poster, storyboard, deck. **Composition** — the HTML/CSS title card, animated on a paused timeline and captured one frame at a time. **Job** — one complete run of the seven stages, identified by a UUID. **Shot** — one prompt, one provider generation, one clip. **Stage** — one step of a job, with its own status and message. **Vault** — the OS credential store holding provider keys, plus the metadata file.

FrameForge Studio 0.1.0 — Guide & Operations Manual